SPI Protocol

Engineering Handbook

STM32F4 • ESP32 • S32K1 • AUTOSAR • KV5x

Full register-level reference with complete bit definitions

Version 2 — Expanded Register Definitions

1. SPI Protocol Fundamentals

1.1 Overview

Serial Peripheral Interface (SPI) is a synchronous, full-duplex serial communication protocol developed by Motorola. It operates in a master-slave architecture and is widely used for connecting microcontrollers to sensors, memories, displays, ADCs, DACs, and other peripherals.
SPI uses four primary signals:

- SCLK (Serial Clock) — Generated by the master; synchronizes data transfer.
- MOSI (Master Out, Slave In) — Data driven by the master and received by slaves.
- MISO (Master In, Slave Out) — Data driven by the selected slave and received by the master.
- CS / NSS (Chip Select / Slave Select) — Active-low line managed by master to select a specific slave.

1.2 Clock Polarity (CPOL) and Clock Phase (CPHA)

SPI defines four modes based on two parameters:

Mode	CPOL	CPHA	Clock Idle	Data Capture Edge	Data Shift Edge
Mode 0	0	0	Low	Rising	Falling
Mode 1	0	1	Low	Falling	Rising
Mode 2	1	0	High	Falling	Rising
Mode 3	1	1	High	Rising	Falling

> *Most devices default to Mode 0 (CPOL=0, CPHA=0). Always verify the peripheral datasheet before configuring the clock mode.*

1.3 Data Framing and Transfer Mechanics

SPI is a shift-register-based protocol. On each clock edge, one bit is shifted out on MOSI and one bit is shifted in on MISO simultaneously. A complete word transfer (typically 8 or 16 bits) requires 8 or 16 clock cycles. The CS line must be asserted (driven low) before the first clock edge and deasserted after the last clock edge.
SPI supports several bus topologies:

- Independent Slaves — Each slave has its own dedicated CS line (most common).
- Daisy-Chain — MISO of one device connects to MOSI of the next; a single CS enables all.
- Shared Bus — Multiple masters, requiring arbitration logic.

1.4 SPI vs Other Protocols

Feature	SPI	I2C	UART
Topology	1-to-N (dedicated CS)	Multi-master bus	Point-to-point
Wires	4 (SCLK, MOSI, MISO, CS)	2 (SDA, SCL)	2 (TX, RX)
Max Speed	50+ MHz typical	3.4 MHz (HS)	~20 Mbps async
Full-Duplex	Yes	No	Yes

Hardware Overhead	Low	ACK/NACK required	Simple framing
Address Needed	No (CS selects slave)	7/10-bit address	N/A

2. SPI on STM32F4 (bxSPI)

2.1 Hardware Overview

STM32F4 provides up to 6 SPI peripherals. SPI1/SPI4 reside on APB2 (up to 42 MHz); SPI2/SPI3 on APB1 (up to 21 MHz). All support 8/16-bit frames, hardware NSS, DMA, TI mode, I2S dual-use, and hardware CRC.

2.2 Complete Register Bit Definitions

SPI_CR1 — Control Register 1

Bit(s)	Name	Reset	Description
15	`BIDIMODE`	0	Bidirectional data mode enable. 0: 2-line unidirectional; 1: 1-line bidirectional.
14	`BIDIOE`	0	Output enable in bidirectional mode. 0: receive-only; 1: transmit-only. Only relevant when BIDIMODE=1.
13	`CRCEN`	0	Hardware CRC calculation enable. 0: disabled; 1: CRC enabled. Must be set before enabling the SPI.
12	`CRCNEXT`	0	Transmit CRC next. 0: data phase; 1: next transfer is CRC. Set after last data byte to send CRC.
11	`DFF`	0	Data frame format. 0: 8-bit data frame; 1: 16-bit data frame.
10	`RXONLY`	0	Receive only mode. 0: full-duplex; 1: output disabled, receive only. Used with BIDIMODE=0.
9	`SSM`	0	Software slave management. 0: NSS pin hardware managed; 1: NSS managed by SSI bit in software.
8	`SSI`	0	Internal slave select. Only used when SSM=1. This bit replaces the NSS pin value.
7	`LSBFIRST`	0	Frame format. 0: MSB transmitted first; 1: LSB transmitted first.
6	`SPE`	0	SPI enable. 0: SPI disabled; 1: SPI enabled. Configuration registers must be set before enabling.
5:3	`BR[2:0]`	000	Baud rate control. Divides APB clock: 000=/2, 001=/4, 010=/8, 011=/16, 100=/32, 101=/64, 110=/128, 111=/256.
2	`MSTR`	0	Master selection. 0: slave configuration; 1: master configuration. NSS must be high before setting this bit.
1	`CPOL`	0	Clock polarity. 0: CK is 0 when idle; 1: CK is 1 when idle.
0	`CPHA`	0	Clock phase. 0: first clock transition is first data capture edge; 1: second clock transition is first capture edge.

SPI_CR2 — Control Register 2

Bit(s)	Name	Reset	Description
7	`TXEIE`	0	TX buffer empty interrupt enable. 0: masked; 1: interrupt when TXE=1.
6	`RXNEIE`	0	RX buffer not empty interrupt enable. 0: masked; 1: interrupt when RXNE=1.
5	`ERRIE`	0	Error interrupt enable. 0: masked; 1: interrupt on CRCERR, OVR, MODF (UDR in slave mode).
4	`FRF`	0	Frame format. 0: SPI Motorola mode; 1: SPI TI mode.
2	`SSOE`	0	SS output enable. 0: SS output disabled (multi-master); 1: SS output enabled in master mode. Cannot be used in multi-master.
1	`TXDMAEN`	0	TX buffer DMA enable. 0: disabled; 1: DMA request when TXE=1.
0	`RXDMAEN`	0	RX buffer DMA enable. 0: disabled; 1: DMA request when RXNE=1.

SPI_SR — Status Register

Bit(s)	Name	Reset	Description
8	`FRE`	0	Frame format error. Set by hardware when TI mode is selected and NSS pulse is missing. Cleared by reading SR.
7	`BSY`	0	Busy flag (read-only). 0: SPI not busy; 1: SPI is busy communicating or TX buffer not empty. Do not disable SPI while BSY=1.
6	`OVR`	0	Overrun flag. Set when data received while RXNE was not yet cleared. Cleared by reading DR then SR.
5	`MODF`	0	Mode fault. Set when NSS goes low in master mode (multi-master conflict). Cleared by reading SR then writing CR1.
4	`CRCERR`	0	CRC error flag. Set when received CRC does not match SPI_RXCRCR. Cleared by software writing 0.
3	`UDR`	0	Underrun flag (slave mode only). Set when clock received but no data was loaded into DR. Cleared by reading DR then SR.
2	`CHSIDE`	0	Channel side (I2S mode). 0: left channel; 1: right channel.
1	`TXE`	1	Transmit buffer empty. 0: not empty; 1: transmit buffer empty, ready to accept next byte.
0	`RXNE`	0	Receive buffer not empty. 0: empty; 1: received data available in DR.

SPI_DR — Data Register

Bit(s)	Name	Reset	Description
15:0	`DR[15:0]`	0x0000	Data register. Write to load TX FIFO; read to empty RX FIFO. For 8-bit mode access as uint8_t to avoid FIFO misalignment. For 16-bit mode access as uint16_t.

SPI_CRCPR — CRC Polynomial Register

Bit(s)	Name	Reset	Description
15:0	`CRCPOLY[15:0]`	0x0007	CRC polynomial register. Default polynomial is 0x0007 (x^7+x^2+x+1 for 8-bit, lower 8 bits used). Must be configured before CRC is enabled.

SPI_RXCRCR — RX CRC Register

Bit(s)	Name	Reset	Description
15:0	`RxCRC[15:0]`	0x0000	Read-only. Contains the CRC calculated from received bytes when CRCEN=1. Compared against the received CRC byte automatically.

SPI_TXCRCR — TX CRC Register

Bit(s)	Name	Reset	Description
15:0	`TxCRC[15:0]`	0x0000	Read-only. Contains the CRC computed from transmitted bytes when CRCEN=1. Transmitted automatically after CRCNEXT is set.

SPI_I2SCFGR — I2S Configuration Register

Bit(s)	Name	Reset	Description
11	`I2SMOD`	0	I2S mode selection. 0: SPI mode; 1: I2S mode selected. When set, I2S configuration bits control operation instead of SPI_CR1/CR2.
10	`I2SE`	0	I2S enable. 0: I2S peripheral disabled; 1: enabled.
9:8	`I2SCFG[1:0]`	00	I2S configuration mode. 00: slave transmit; 01: slave receive; 10: master transmit; 11: master receive.
7	`PCMSYNC`	0	PCM frame synchronisation. 0: short frame sync; 1: long frame sync.
5:4	`I2SSTD[1:0]`	00	I2S standard selection. 00: Philips; 01: MSB justified; 10: LSB justified; 11: PCM standard.
3	`CKPOL`	0	Steady state clock polarity. 0: low; 1: high.
2:1	`DATLEN[1:0]`	00	Data length. 00: 16-bit; 01: 24-bit; 10: 32-bit; 11: not allowed.

0	**CHLEN**	0	Channel length. 0: 16-bit per audio channel; 1: 32-bit per audio channel.

2.3 Clock Prescaler (BR[2:0]) Table

BR[2:0]	Divisor	SPI1/4 @ 84 MHz APB2	SPI2/3 @ 42 MHz APB1
000	2	42.0 MHz	21.0 MHz
001	4	21.0 MHz	10.5 MHz
010	8	10.5 MHz	5.25 MHz
011	16	5.25 MHz	2.63 MHz
100	32	2.63 MHz	1.31 MHz
101	64	1.31 MHz	656 kHz
110	128	656 kHz	328 kHz
111	256	328 kHz	164 kHz

2.4 HAL Initialization Example

```
SPI_HandleTypeDef hspi1;
void MX_SPI1_Init(void) {
    hspi1.Instance               = SPI1;
    hspi1.Init.Mode              = SPI_MODE_MASTER;
    hspi1.Init.Direction         = SPI_DIRECTION_2LINES;
    hspi1.Init.DataSize          = SPI_DATASIZE_8BIT;
    hspi1.Init.CLKPolarity       = SPI_POLARITY_LOW;
    hspi1.Init.CLKPhase          = SPI_PHASE_1EDGE;
    hspi1.Init.NSS               = SPI_NSS_SOFT;
    hspi1.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_8;
    hspi1.Init.FirstBit          = SPI_FIRSTBIT_MSB;
    hspi1.Init.TIMode            = SPI_TIMODE_DISABLE;
    hspi1.Init.CRCCalculation    = SPI_CRCCALCULATION_DISABLE;
    HAL_SPI_Init(&hspi1);
}

/* Polling transfer */
uint8_t tx[4]={0x01,0x02,0x03,0x04}, rx[4];
HAL_GPIO_WritePin(CS_GPIO_Port,CS_Pin,GPIO_PIN_RESET);
HAL_SPI_TransmitReceive(&hspi1,tx,rx,4,HAL_MAX_DELAY);
HAL_GPIO_WritePin(CS_GPIO_Port,CS_Pin,GPIO_PIN_SET);
```

2.5 Direct Register Access

```
/* Enable SPI1, GPIOA */
RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
/* PA5=SCK, PA6=MISO, PA7=MOSI as AF5 */
GPIOA->MODER  |= (2<<10)|(2<<12)|(2<<14);
GPIOA->AFR[0] |= (5<<20)|(5<<24)|(5<<28);
/* Master, CPOL=0, CPHA=0, BR=/8, SSM+SSI */
```

```
SPI1->CR1 = SPI_CR1_MSTR|SPI_CR1_SSM|SPI_CR1_SSI|(2<<SPI_CR1_BR_Pos);
SPI1->CR1 |= SPI_CR1_SPE;

uint8_t spi_xfer(uint8_t d){
    while(!(SPI1->SR & SPI_SR_TXE));
    *(volatile uint8_t*)&SPI1->DR = d;
    while(!(SPI1->SR & SPI_SR_RXNE));
    return *(volatile uint8_t*)&SPI1->DR;
}
```

Always access SPI_DR as uint8_t (not uint16_t) in 8-bit mode to prevent inadvertently reading the shift register and misaligning the FIFO.

3. SPI on ESP32 (GPSPI)

3.1 Hardware Overview

ESP32 provides four SPI controllers. SPI0/SPI1 are reserved for internal Flash/PSRAM. SPI2 (HSPI) and SPI3 (VSPI) are user-accessible. Each supports up to 80 MHz, full/half-duplex, DMA, four hardware CS lines, per-transaction configuration, and command/address phases.

3.2 ESP32 SPI Hardware Registers — Full Bit Definitions

ESP32 SPI registers are memory-mapped. The following tables cover the most important registers for SPI2 (base 0x3FF64000) and SPI3 (base 0x3FF65000).

SPI_CMD — Command Register (offset 0x00)

Bit(s)	Name	Reset	Description
31	`SPI_USR`	0	Set to 1 to trigger an SPI transfer. Cleared by hardware when transfer is complete. Poll this bit to detect transfer done in polling mode.

SPI_ADDR — Address Register (offset 0x04)

Bit(s)	Name	Reset	Description
31:0	`SPI_USR_ADDR_VALUE`	0	Address value to transmit during the address phase (if SPI_USR_ADDR=1 in SPI_USER). The address is sent MSB first. The actual number of bits sent is configured by SPI_USR_ADDR_BITLEN in SPI_USER1.

SPI_CTRL — Control Register (offset 0x08)

Bit(s)	Name	Reset	Description
25	`SPI_WR_BIT_ORDER`	0	Write bit order. 0: MSB first on MOSI; 1: LSB first on MOSI.
24	`SPI_RD_BIT_ORDER`	0	Read bit order. 0: MSB first from MISO; 1: LSB first from MISO.
13	`SPI_QIO_MODE`	0	In quad I/O mode, both address and read data phases use 4 data lines. Sets MOSI/MISO/WP/HD as bidirectional quad lines.
12	`SPI_DIO_MODE`	0	In dual I/O mode, address and data phases use 2 data lines (MOSI+MISO bidirectional).
10	`SPI_FASTRD_MODE`	1	Enable fast read mode. When 0 forces single-bit SPI regardless of FREAD settings.
7	`SPI_FREAD_QIO`	0	Fast read quad I/O mode. Uses 4 lines for both address and read data.

6	SPI_FREAD_DIO	0	Fast read dual I/O mode. Uses 2 lines for both address and read data.
3	SPI_FREAD_QUAD	0	Fast read quad output. Uses 4 lines for data output only.
2	SPI_FREAD_DUAL	0	Fast read dual output. Uses 2 lines (MOSI+MISO) for data output only.

SPI_USER — User Register (offset 0x1C) — Core transfer config

Bit(s)	Name	Reset	Description
31	SPI_USR_COMMAND	1	Enable command phase. A 1–16 bit command is sent before any other phase. Command value set in SPI_USER2.SPI_USR_COMMAND_VALUE.
30	SPI_USR_ADDR	0	Enable address phase. The address in SPI_ADDR is transmitted. Length set by SPI_USR_ADDR_BITLEN in SPI_USER1.
29	SPI_USR_DUMMY	0	Enable dummy phase. Inserts clock cycles with no data (for read latency). Cycle count set by SPI_USR_DUMMY_CYCLELEN in SPI_USER1.
28	SPI_USR_MISO	0	Enable MISO (read) data phase. Master reads data from slave on MISO. Length set by SPI_USR_MISO_DBITLEN in SPI_USER1.
27	SPI_USR_MOSI	0	Enable MOSI (write) data phase. Master writes data to slave on MOSI. Length set by SPI_USR_MOSI_DBITLEN in SPI_USER1.
26	SPI_USR_MOSI_HIGHPART	0	MOSI data from high part of SPI_W8–15 registers (instead of SPI_W0–7).
25	SPI_USR_MISO_HIGHPART	0	MISO data stored in high part of SPI_W8–15 registers.
24	SPI_USR_PREP_HOLD	0	Hold CS asserted during preparation phase.
23	SPI_USR_CMD_HOLD	0	Hold CS asserted at end of command phase.
22	SPI_USR_ADDR_HOLD	0	Hold CS asserted at end of address phase.
21	SPI_USR_DUMMY_HOLD	0	Hold CS asserted at end of dummy phase.
20	SPI_USR_DIN_HOLD	0	Hold CS asserted at end of MISO phase.
19	SPI_USR_DOUT_HOLD	0	Hold CS asserted at end of MOSI phase.
18	SPI_USR_HOLD_POL	0	CS hold polarity. 0: CS is high during hold; 1: CS is low during hold.
16	SPI_SIO	0	Single-line bidirectional I/O mode (3-wire). MOSI is used for both sending and receiving.

15	`SPI_FWRITE_QIO`	0	Write in quad I/O mode. 4-bit data output using MOSI/MISO/WP/HD.
14	`SPI_FWRITE_DIO`	0	Write in dual I/O mode. 2-bit data output using MOSI/MISO.
13	`SPI_FWRITE_QUAD`	0	Write in quad output mode.
12	`SPI_FWRITE_DUAL`	0	Write in dual output mode.
11	`SPI_WR_BYTE_ORDER`	0	MOSI byte order. 0: byte N from W[N/4][byte] (little-endian); 1: big-endian byte ordering.
10	`SPI_RD_BYTE_ORDER`	0	MISO byte order. 0: little-endian storage; 1: big-endian storage.
9:8	`SPI_CK_OUT_EDGE / SPI_CK_I_EDGE`	0	Combined with SPI_CTRL2 to configure CPOL/CPHA. See clock mode table.
7	`SPI_CS_SETUP`	0	Enable CS setup time. CS is asserted before the first clock edge. Setup cycle count in SPI_CTRL2.
6	`SPI_CS_HOLD`	0	Enable CS hold time. CS stays asserted after the last clock edge. Hold cycle count in SPI_CTRL2.
5	`SPI_CK_OUT_EDGE`	0	(See bits 9:8.) Adjusts clock phase in conjunction with SPI_CK_I_EDGE.
4	`SPI_CK_I_EDGE`	1	Clock input edge. Default 1. Combined with SPI_CK_OUT_EDGE to select SPI mode 0/1/2/3.
3	`SPI_DOUTDIN`	0	Full-duplex mode. 0: half-duplex (MOSI only or MISO only per phase); 1: full-duplex simultaneous MOSI/MISO.

SPI_USER1 — User Register 1 (offset 0x20)

Bit(s)	Name	Reset	Description
31:26	`SPI_USR_ADDR_BITLEN[5:0]`	0	Address phase bit length minus 1. E.g., 23 = 24-bit address. Max 63 bits.
24:17	`SPI_USR_MISO_DBITLEN[7:0]`	0	MISO data phase length minus 1 (number of bits to receive - 1). Max 511 bits.
8:0	`SPI_USR_MOSI_DBITLEN[8:0]`	0	MOSI data phase length minus 1 (number of bits to send - 1). Max 511 bits.
16:11	`SPI_USR_DUMMY_CYCLELEN[5:0]`	0	Number of dummy clock cycles minus 1. Only used when SPI_USR_DUMMY=1 in SPI_USER.

SPI_PIN — Pin Control Register (offset 0x34)

Bit(s)	Name	Reset	Description

30	SPI_CS2_DIS	1	Disable CS2 pin. 0: CS2 active; 1: CS2 inactive (default).
29	SPI_CS1_DIS	1	Disable CS1 pin. 0: CS1 active; 1: CS1 inactive (default).
28	SPI_CS0_DIS	0	Disable CS0 pin. 0: CS0 active (default); 1: CS0 inactive.
7	SPI_MASTER_CS_POL	0	Polarity of CS in master mode per CS line. Bit 7=CS0, bit 8=CS1, bit 9=CS2. 0: active low; 1: active high.
5	SPI_CK_DIS	0	Disable SCLK output. 0: clock enabled; 1: clock pin disabled (tri-state).
4	SPI_MASTER_CK_SEL	0	Master clock source select. Normally 0.
2	SPI_CK_IDLE_EDGE	0	SCLK idle level. 0: SCLK is low when idle (CPOL=0); 1: SCLK is high when idle (CPOL=1).
1	SPI_MISO_DELAY_MODE	0	MISO input delay mode. 0: no delay; 1: delay by SPI_MISO_DELAY_NUM APB cycles.

SPI_CLOCK — Clock Register (offset 0x18)

Bit(s)	Name	Reset	Description
31	SPI_CLK_EQU_SYSCLK	1	When 1, SPI clock = system clock (80 MHz max). When 0, clock is divided using CLKDIV_PRE and CLKCNT fields.
21:18	SPI_CLKDIV_PRE[3:0]	0	Clock pre-divider. SPI clock = APB / (CLKDIV_PRE+1) / (CLKCNT_N+1).
17:12	SPI_CLKCNT_N[5:0]	0	Clock duty cycle total. Bit rate = APB / (CLKDIV_PRE+1) / (CLKCNT_N+1).
11:6	SPI_CLKCNT_H[5:0]	0	Clock high period count. Number of APB cycles the clock is high within one SPI clock period.
5:0	SPI_CLKCNT_L[5:0]	0	Clock low period count (= CLKCNT_N typically). Number of APB cycles the clock is low.

3.3 ESP-IDF API Example

```
#include "driver/spi_master.h"
spi_device_handle_t spi;
void spi_init(void){
    spi_bus_config_t bus={.mosi_io_num=23,.miso_io_num=19,
        .sclk_io_num=18,.quadwp_io_num=-1,.quadhd_io_num=-1,.max_transfer_sz=4096};
    spi_device_interface_config_t dev={
        .clock_speed_hz=10000000,.mode=0,.spics_io_num=5,.queue_size=7};
    spi_bus_initialize(SPI2_HOST,&bus,SPI_DMA_CH_AUTO);
    spi_bus_add_device(SPI2_HOST,&dev,&spi);
}
uint8_t spi_byte(uint8_t d){
    spi_transaction_t t={.length=8,.tx_buffer=&d,.flags=SPI_TRANS_USE_RXDATA};
```

```
        spi_device_polling_transmit(spi,&t);
        return t.rx_data[0];
    }
```

4. SPI on S32K1 (LPSPI)

4.1 Hardware Overview

The S32K1xx Low Power SPI (LPSPI) peripheral supports master/slave, 1–4096-bit frames, 4-entry TX/RX FIFOs, programmable delays, 4 PCS lines, and DMA. It requires the module to be disabled (MEN=0) for most configuration changes.

4.2 Complete Register Bit Definitions

LPSPI_CR — Control Register

Bit(s)	Name	Reset	Description
9	`DBGEN`	0	Debug enable. 0: LPSPI disabled in debug mode; 1: LPSPI enabled in debug mode (continues transfers while halted).
3	`DOZEN`	0	Doze mode enable. 0: LPSPI enabled in doze mode; 1: LPSPI disabled in doze mode (low-power idle state).
1	`RST`	0	Software reset. Write 1 to reset all internal logic and registers (except CR itself). Self-clearing after one cycle.
0	`MEN`	0	Module enable. 0: LPSPI disabled; 1: LPSPI enabled. Must be 0 to change most configuration registers. Set last after all config is complete.

LPSPI_SR — Status Register

Bit(s)	Name	Reset	Description
13	`MBF`	0	Module busy flag (read-only). 0: LPSPI idle; 1: LPSPI is busy with a transfer. Check before disabling module.
11	`DMF`	0	Data match flag. Set when received data matches the LPSPI_DMATCH value (if data match is configured). Write 1 to clear.
10	`REF`	0	Receive error flag. Set when RX FIFO overflows (data received when FIFO full). Write 1 to clear.
9	`TEF`	0	Transmit error flag. Set when TX FIFO underruns in slave mode (clock received but no data ready). Write 1 to clear.
8	`TCF`	0	Transfer complete flag. Set when all transfers in a frame are complete (CS deasserted). Write 1 to clear.
7	`FCF`	0	Frame complete flag. Set at end of each frame (each TCR-defined word). Write 1 to clear.
6	`WCF`	0	Word complete flag. Set each time a word transfer is complete. Write 1 to clear.

5	`SRF`	0	Slave ready flag (slave mode). 0: slave not ready; 1: slave ready for transfer.
1	`TDF`	1	Transmit data flag. 0: TX FIFO full (not ready); 1: TX FIFO not full, ready to accept data. Default 1 at power-on.
0	`RDF`	0	Receive data flag. 0: RX FIFO empty; 1: RX FIFO not empty, data ready to read.

LPSPI_IER — Interrupt Enable Register

Bit(s)	Name	Reset	Description
11	`DMIE`	0	Data match interrupt enable. 1: interrupt when DMF=1.
10	`REIE`	0	Receive error interrupt enable. 1: interrupt when REF=1.
9	`TEIE`	0	Transmit error interrupt enable. 1: interrupt when TEF=1.
8	`TCIE`	0	Transfer complete interrupt enable. 1: interrupt when TCF=1.
7	`FCIE`	0	Frame complete interrupt enable. 1: interrupt when FCF=1.
6	`WCIE`	0	Word complete interrupt enable. 1: interrupt when WCF=1.
1	`TDIE`	0	Transmit data interrupt enable. 1: interrupt when TDF=1 (TX FIFO not full).
0	`RDIE`	0	Receive data interrupt enable. 1: interrupt when RDF=1 (RX FIFO not empty).

LPSPI_FCR — FIFO Control Register

Bit(s)	Name	Reset	Description
19:16	`RXWATER[3:0]`	0	RX FIFO watermark. RDF is set when the number of words in the RX FIFO is greater than RXWATER. Range 0–3 for 4-entry FIFO.
3:0	`TXWATER[3:0]`	0	TX FIFO watermark. TDF is set when the number of words in TX FIFO is less than or equal to TXWATER. Range 0–3.

LPSPI_FSR — FIFO Status Register

Bit(s)	Name	Reset	Description
19:16	`RXCOUNT[3:0]`	0	Read-only. Current number of words in the RX FIFO (0–4).
3:0	`TXCOUNT[3:0]`	0	Read-only. Current number of words in the TX FIFO (0–4).

LPSPI_TCR — Transmit Command Register

Bit(s)	Name	Reset	Description

31	`CPOL`	0	Clock polarity. 0: SCK is low when PCS is asserted (idle low); 1: SCK is high when PCS is asserted (idle high).
30	`CPHA`	0	Clock phase. 0: data captured on leading edge, changed on trailing edge; 1: data changed on leading edge, captured on trailing edge.
29:27	`PRESCALE[2:0]`	0	Baud rate prescaler. SCK = (LPSPI functional clock) / 2^PRESCALE / (SCKDIV+2). 000=/1, 001=/2, 010=/4, ..., 111=/128.
25:24	`PCS[1:0]`	0	Peripheral chip select. Selects which PCS line to assert: 00=PCS0, 01=PCS1, 10=PCS2, 11=PCS3.
21	`LSBF`	0	LSB first. 0: MSB transmitted and received first; 1: LSB first.
20	`BYSW`	0	Byte swap. 0: no swap; 1: byte-swap each 32-bit word before transmit (affects endianness of multi-byte frames).
19	`CONT`	0	Continuous transfer. 0: PCS deasserted between each word; 1: PCS stays asserted across consecutive words (continuous chip select).
18	`CONTC`	0	Continuing command. 0: new TCR command; 1: continuation of previous command (PCS kept asserted from last frame).
17	`RXMSK`	0	Receive data mask. 0: received data stored in RX FIFO; 1: received data discarded (write-only transfer).
16	`TXMSK`	0	Transmit data mask. 0: data loaded from TX FIFO; 1: transmit tri-stated — no data sent (read-only transfer with dummy clocks).
11:0	`FRAMESZ[11:0]`	0x00F	Frame size minus 1. Total bits in one SPI frame = FRAMESZ+1. Range 0–4095 (1–4096 bits). Default 0x00F = 16-bit frame.

LPSPI_CCR — Clock Configuration Register

Bit(s)	Name	Reset	Description
31:24	`SCKPCS[7:0]`	0	SCK-to-PCS delay. Number of cycles of the LPSPI functional clock between the last SCK edge and PCS deassertion. Minimum 1 cycle.
23:16	`PCSSCK[7:0]`	0	PCS-to-SCK delay. Number of functional clock cycles from PCS assertion to first SCK edge. Minimum 1 cycle.
15:8	`DBT[7:0]`	0	Delay between transfers. Number of functional clock cycles inserted between consecutive transfers (between CS deassertion of one and CS assertion of next).
7:0	`SCKDIV[7:0]`	0	SCK divider. SCK period = 2 * (SCKDIV + 2) / (functional clock). Together with PRESCALE in TCR sets the final baud rate.

LPSPI_CFGR0 — Configuration Register 0

Bit(s)	Name	Reset	Description
9	`RDMO`	0	Receive data match only. 0: normal operation; 1: only frames matching LPSPI_DMATCH are stored in RX FIFO.
8	`CIRFIFO`	0	Circular FIFO enable. 0: normal FIFO; 1: TX FIFO operates as circular buffer (last word retransmitted when FIFO empties).
2	`HRSEL`	0	Host request select. 0: host request input from HREQ pin; 1: from internal trigger source.
1	`HRPOL`	0	Host request polarity. 0: active high; 1: active low.
0	`HREN`	0	Host request enable. When enabled, LPSPI only transmits when host request signal is active.

LPSPI_CFGR1 — Configuration Register 1

Bit(s)	Name	Reset	Description
27	`PCSCFG`	0	PCS configuration. 0: PCS[3:2] are normal chip-select outputs; 1: PCS[3:2] are used as SCLK/LPSPI_SIN (half-duplex 2-wire mode).
26	`OUTCFG`	0	Output configuration. 0: output data retains last value between frames; 1: output data tri-stated between frames.
25:24	`PINCFG[1:0]`	0	Pin configuration. 00: SDO=MOSI, SDI=MISO; 01: SDO=MOSI, SDI=MOSI (SDI unused); 10: SDO=MISO, SDI=MOSI (reversed); 11: SDO=MISO, SDI=MISO.
3	`AUTOPCS`	0	Automatic PCS. 0: manual PCS via TCR; 1: PCS assertion controlled automatically by hardware.
2	`NOSTALL`	0	No stall. 0: stall clock if TX FIFO empty or RX FIFO full; 1: do not stall (may cause underrun/overrun).
1	`SAMPLE`	0	Sample point. 0: input data sampled on SCK edge; 1: input data sampled one LPSPI functional clock cycle after SCK edge (for high-speed).
0	`MASTER`	0	Master mode. 0: slave mode; 1: master mode. Must be written when MEN=0.

4.3 SDK Usage Example

```
#include "lpspi_master_driver.h"
lpspi_state_t state;
const lpspi_master_config_t cfg = {
    .bitsPerSec=1000000U, .whichPcs=LPSPI_PCS0,
    .pcsPolarity=LPSPI_ACTIVE_LOW, .bitcount=8U,
    .clkPhase=LPSPI_CLOCK_PHASE_1ST_EDGE,
    .clkPolarity=LPSPI_SCK_ACTIVE_HIGH,
    .lsbFirst=false,
    .transferType=LPSPI_USING_INTERRUPTS,
```

```
        .rxDMAChannel=255U, .txDMAChannel=255U,
    };
    LPSPI_DRV_MasterInit(LPSPICOM1,&state,&cfg);

    uint8_t tx[4]={0xDE,0xAD,0xBE,0xEF}, rx[4];
    LPSPI_DRV_MasterTransferBlocking(LPSPICOM1,tx,rx,4U,100U);
```

5. SPI in AUTOSAR

5.1 Architecture Overview

AUTOSAR SPI (Spi module, MCAL layer) abstracts hardware SPI through three concepts: Channels (data buffers), Jobs (sequences of channels sharing one CS), and Sequences (ordered job sets). The driver generates status flags, interrupt callbacks, and DMA control through a standardised API.

5.2 AUTOSAR Spi Module — API and Status Types

Type / Value	Meaning
SPI_UNINIT	Driver not initialised. Spi_Init() not yet called.
SPI_IDLE	Driver initialised, no transfer in progress.
SPI_BUSY	At least one sequence transfer is ongoing.
SPI_SEQ_OK	Last sequence transfer completed successfully.
SPI_SEQ_PENDING	Sequence transfer is ongoing (not yet complete).
SPI_SEQ_FAILED	Sequence transfer failed (error occurred).
SPI_SEQ_CANCELLED	Sequence was cancelled via Spi_Cancel().
SPI_JOB_OK	Last job within a sequence completed successfully.
SPI_JOB_PENDING	Job transfer is ongoing.
SPI_JOB_FAILED	Job transfer failed.
SPI_JOB_QUEUED	Job is queued, waiting for bus.
SPI_IB	Internal Buffer channel type — data copied to internal driver buffer.
SPI_EB	External Buffer channel type — pointer to application-provided buffer.

5.3 Spi Channel Configuration Fields — Full Definition

Field	Type	Description
SpiChannelId	uint8	Unique channel identifier. Used in all API calls to reference this channel.
SpiChannelType	SPI_IB / SPI_EB	IB: data is copied into an internal driver buffer (up to SpiIbNBuffers words). EB: pointer to external app buffer set via Spi_SetupEB at runtime.

SpiDataWidth	1–32 bits	Width of each data element in bits. Determines how many bits are shifted per element.
SpiDefaultData	uint32	Default data used for transmit when tx_buffer is NULL (receive-only transfers). Also initial FIFO fill value.
SpiEbMaxLength	uint16	Maximum number of data elements for External Buffer mode. Checked by Spi_SetupEB to prevent overflow.
SpiIbNBuffers	uint16	Number of data elements in the internal buffer for IB channels.
SpiTransferStart	SPI_MSB / SPI_LSB	Bit order: SPI_MSB = most significant bit first; SPI_LSB = least significant bit first.

5.4 Spi Job Configuration Fields — Full Definition

Field	Type	Description
SpiJobId	uint8	Unique job identifier.
SpiHwUnit	SpiExternalDeviceType	Reference to the external SPI device (hardware unit + CS + bit timing) this job uses.
SpiJobPriority	0–3	Job scheduling priority. 0 = lowest, 3 = highest. Higher-priority jobs preempt lower ones when async scheduler is used.
SpiJobEndNotification	fn ptr	Callback function invoked when the job completes (or NULL if not needed). Called from ISR context — keep short.
SpiCsIdentifier	enum	Chip select identifier — maps to a hardware CS line or a GPIO port/pin for software CS.
SpiCsPolarity	STD_HIGH/LOW	Active polarity of the chip select line. STD_LOW = active low (most common).
SpiCsCallback	fn ptr	Optional callback for software-controlled CS assertion/deassertion. Called before first and after last channel transfer.
ChannelList[]	uint8[]	Ordered list of channel IDs included in this job. Channels are transferred in order.
ChannelCount	uint8	Number of entries in ChannelList[].

5.5 Spi Sequence Configuration Fields — Full Definition

Field	Type	Description
SpiSequenceId	uint8	Unique sequence identifier. Used in Spi_AsyncTransmit / Spi_SyncTransmit.
SpiInterruptibleSequence	bool	FALSE: sequence runs atomically (cannot be interrupted by a higher-priority sequence). TRUE: may be interleaved.
SpiSeqEndNotification	fn ptr	Callback invoked when the entire sequence completes. Called from ISR/scheduler context.
JobList[]	uint8[]	Ordered list of job IDs in this sequence.
JobCount	uint8	Number of jobs in JobList[].

5.6 External Device Configuration Fields

Field	Type	Description
SpiBaudrate	uint32	SCK frequency in Hz for this device. The driver selects the closest achievable hardware prescaler.
SpiCsPolarity	STD_HIGH/LOW	Chip select polarity for this device.
SpiCsIdentifier	enum	Which CS line (hardware or GPIO) is used for this device.
SpiDataShiftEdge	LEADING/TRAILING	Which SCK edge data is shifted (output). Combines with SpiTimeClkBusIdle to define the SPI mode.
SpiEnableCs	bool	TRUE: driver controls CS automatically; FALSE: application controls CS manually.
SpiHwUnit	uint8	Reference to the physical SPI hardware unit (e.g., SpiHwUnit_0 = SPI0).
SpiShiftClockIdleLevel	LOW/HIGH	SCK level when idle (CPOL). LOW=CPOL0, HIGH=CPOL1.
SpiTimeClkBusIdle	float	Time (ns) the clock is inactive between transfers. Used to configure inter-word delay.
SpiTimeBetweenClk	float	Time (ns) of one clock period. Inversely proportional to SpiBaudrate.
SpiTimeCs2Clk	float	CS-to-SCK setup time in ns (CS asserted to first clock edge).

SpiTimeClk2Cs	float	SCK-to-CS hold time in ns (last clock edge to CS deassertion).

6. SPI on Kinetis KV5x (DSPI)

6.1 Hardware Overview

KV5x DSPI supports 4–16-bit frames, 6 hardware PCS lines, 4-entry TX/RX FIFOs, continuous SCK mode, modified transfer format, EOQ (end-of-queue) signalling, and eDMA. Each CTAR (Clock and Transfer Attributes Register) independently configures baud, delays, and frame format.

6.2 Complete Register Bit Definitions

DSPI_MCR — Module Configuration Register

Bit(s)	Name	Reset	Description
31	`MSTR`	0	Master/slave mode select. 0: slave mode; 1: master mode. Change only while HALT=1.
30	`CONT_SCKE`	0	Continuous SCK enable. 0: SCK inactive between transfers; 1: SCK runs continuously. Useful for devices requiring continuous clock.
29:28	`DCONF[1:0]`	00	DSPI configuration. 00: SPI; 01: DSI; 10: CSI; 11: reserved. Always 00 for standard SPI.
27	`FRZ`	0	Freeze. 0: normal operation; 1: halt transfers when entering debug mode.
26	`MTFE`	0	Modified transfer format enable. 0: standard SPI; 1: modified format (CS asserted for entire queue).
25	`PCSSE`	0	Peripheral chip select strobe enable. 0: PCS5 is a normal CS; 1: PCS5 acts as an active-low strobe at end of transfer.
24	`ROOE`	0	Receive FIFO overflow overwrite enable. 0: incoming data is ignored when RX FIFO full; 1: incoming data overwrites oldest entry.
20:18	`PCSIS[5:0]`	0	PCS inactive state. One bit per PCS line (bits 20:18 = PCS5:0 respectively, across MCR bits 20–15). 0: PCS inactive high; 1: PCS inactive low.
17	`DOZE`	0	Doze enable. 0: DSPI runs in doze mode; 1: DSPI halted in doze mode.
16	`MDIS`	0	Module disable. 0: DSPI enabled; 1: DSPI clock gated off (low power). Must be 0 for any transfer.
13	`DIS_TXF`	0	Disable TX FIFO. 0: TX FIFO enabled (4 entries); 1: TX FIFO bypassed (single-entry mode). Useful for tight timing.
12	`DIS_RXF`	0	Disable RX FIFO. 0: RX FIFO enabled; 1: RX FIFO bypassed.
11	`CLR_TXF`	0	Clear TX FIFO. Write 1 to flush all entries. Self-clearing.

10	`CLR_RXF`	0	Clear RX FIFO. Write 1 to flush all entries. Self-clearing.
9:8	`SMPL_PT[1:0]`	00	Sample point. Controls when MISO is sampled relative to SCK: 00=0 clk, 01=1 clk, 10=2 clks after SCK edge. Use 01/10 at high speeds.
0	`HALT`	1	Halt. 1: transfers halted; 0: transfers run. Default 1 at reset. Clear to start operation.

DSPI_SR — Status Register

Bit(s)	Name	Reset	Description
31	`TCF`	0	Transfer complete flag. Set when all bits of a frame have been shifted and PCS deasserted (or CONT=0). Write 1 to clear.
30	`TXRXS`	0	TX and RX status (read-only). 0: transfer stopped; 1: transfer in progress.
28	`EOQF`	0	End of queue flag. Set when the EOQ bit in PUSHR was set and the last word of a queue has been transferred. Write 1 to clear.
27	`TFUF`	0	TX FIFO underflow flag. Set when TX FIFO is empty and a transfer is requested (slave mode). Write 1 to clear.
25	`TFFF`	1	TX FIFO fill flag. 1: TX FIFO not full, can accept data; 0: TX FIFO full. Used to gate TXDMAEN or TFFFDIRS interrupt.
19	`RFOF`	0	RX FIFO overflow flag. Set when a received word cannot be stored (FIFO full and ROOE=0). Write 1 to clear.
17	`RFDF`	0	RX FIFO drain flag. 1: RX FIFO not empty, data available; 0: empty. Used to gate RXDMAEN or RFDFDIRS interrupt.
15:12	`TXCTR[3:0]`	0	TX FIFO counter (read-only). Number of valid entries currently in the TX FIFO (0–4).
11:8	`TXNXTPTR[3:0]`	0	TX next pointer (read-only). Pointer to next TX FIFO entry to be transferred.
7:4	`RXCTR[3:0]`	0	RX FIFO counter (read-only). Number of valid entries currently in the RX FIFO (0–4).
3:0	`POPNXTPTR[3:0]`	0	RX pop next pointer (read-only). Pointer to next RX FIFO entry to be read via POPR.

DSPI_CTARn — Clock and Transfer Attributes Register (n = 0 or 1)

Bit(s)	Name	Reset	Description
31	`DBR`	0	Double baud rate. 0: baud rate = BUSCLK/(PBR * BR); 1: baud rate = BUSCLK*2/(PBR * BR). Note: 50% duty cycle not guaranteed when DBR=1.

30:27	`FMSZ[3:0]`	0xF	Frame size minus 1. Number of bits per SPI frame = FMSZ+1. Range 3–15 (4–16 bits).
26	`CPOL`	0	Clock polarity. 0: SCK idle low; 1: SCK idle high.
25	`CPHA`	0	Clock phase. 0: data captured on leading edge; 1: data captured on trailing edge.
24	`LSBFE`	0	LSB first enable. 0: MSB first (default); 1: LSB transmitted/received first.
23:22	`PCSSCK[1:0]`	0	PCS to SCK delay prescaler. Combined with CSSCK scaler: t_PCS2SCK = t_BUS * PCSSCK_val * CSSCK_val. 00=2, 01=3, 10=5, 11=7.
21:20	`PASC[1:0]`	0	After SCK delay prescaler. Combined with ASC: t_ASC = t_BUS * PASC_val * ASC_val. 00=2, 01=3, 10=5, 11=7.
19:18	`PDT[1:0]`	0	Delay after transfer prescaler. Combined with DT: t_DT = t_BUS * PDT_val * DT_val. 00=2, 01=3, 10=5, 11=7.
17:16	`PBR[1:0]`	0	Baud rate prescaler. 00=2, 01=3, 10=5, 11=7. Baud = BUSCLK / (PBR * BR * (1+DBR-inverse)).
15:12	`CSSCK[3:0]`	0	PCS to SCK delay scaler. Actual scaler = 2^(CSSCK+1): 0=2, 1=4, 2=8 ... 15=65536.
11:8	`ASC[3:0]`	0	After SCK delay scaler. Same encoding as CSSCK: 2^(ASC+1).
7:4	`DT[3:0]`	0	Delay after transfer scaler. Same encoding: 2^(DT+1).
3:0	`BR[3:0]`	0	Baud rate scaler. Actual scaler = 2^(BR+1): 0=2, 1=4, 2=8 ... 15=65536.

DSPI_PUSHR — Push TX FIFO Register

Bit(s)	Name	Reset	Description
31	`CONT`	0	Continuous PCS assertion. 0: PCS deasserted between transfers; 1: PCS remains asserted for multi-word transfers (burst mode).
30:28	`CTAS[2:0]`	0	Clock and transfer attributes select. Selects which CTARn (0 or 1) governs this transfer's timing and frame size.
27	`EOQ`	0	End of queue marker. 0: more data follows; 1: this is the last word in the transfer queue. Sets EOQF in SR when transferred; also deasserts PCS if CONT=0.
26	`CTCNT`	0	Clear transfer counter. 0: no action; 1: resets DSPI_TCR.SPI_TCNT to 0 before this transfer begins.
21:16	`PCS[5:0]`	0	Peripheral chip select. Bit mask: bit 0=PCS0, bit 1=PCS1, ..., bit 5=PCS5. One or more PCS lines are asserted during the transfer.

15:0	**TXDATA[15:0]**	0	Transmit data. The data to shift out. Valid bits depend on FMSZ+1 in the selected CTAR.

DSPI_POPR — Pop RX FIFO Register

Bit(s)	Name	Reset	Description
31:0	**RXDATA[31:0]**	—	Received data (read-only). Reading this register pops the oldest entry from the RX FIFO. Valid data is in the lower FMSZ+1 bits.

DSPI_RSER — DMA/Interrupt Request Select and Enable Register

Bit(s)	Name	Reset	Description
31	**TCF_RE**	0	Transmission complete request enable. 1: DMA/interrupt request when TCF=1.
28	**EOQF_RE**	0	End of queue request enable. 1: DMA/interrupt request when EOQF=1.
27	**TFUF_RE**	0	TX FIFO underflow request enable. 1: interrupt when TFUF=1.
25	**TFFF_RE**	0	TX FIFO fill request enable. 1: DMA or interrupt request when TFFF=1.
24	**TFFF_DIRS**	0	TX FIFO fill DMA or interrupt select. 0: interrupt; 1: DMA request.
19	**RFOF_RE**	0	RX FIFO overflow request enable. 1: interrupt when RFOF=1.
17	**RFDF_RE**	0	RX FIFO drain request enable. 1: DMA or interrupt when RFDF=1.
16	**RFDF_DIRS**	0	RX FIFO drain DMA or interrupt select. 0: interrupt; 1: DMA request.

6.3 SDK Usage Example

```
#include "fsl_dspi.h"
dspi_master_config_t cfg;
DSPI_MasterGetDefaultConfig(&cfg);
cfg.ctarConfig.baudRate=500000U;
cfg.ctarConfig.bitsPerFrame=8U;
cfg.ctarConfig.cpol=kDSPI_ClockPolarityActiveHigh;
cfg.ctarConfig.cpha=kDSPI_ClockPhaseFirstEdge;
cfg.whichPcs=kDSPI_Pcs0;
DSPI_MasterInit(SPI0,&cfg,CLOCK_GetFreq(kCLOCK_BusClk));

dspi_transfer_t xfer={.txData=txBuf,.rxData=rxBuf,
    .dataSize=4U,.configFlags=kDSPI_MasterCtar0|kDSPI_MasterPcs0};
DSPI_MasterTransferBlocking(SPI0,&xfer);
```

7. Platform Comparison & Best Practices

7.1 Feature Comparison

Feature	STM32F4	ESP32	S32K1 LPSPI	AUTOSAR	KV5x DSPI
Max Speed	42 MHz	80 MHz	48 MHz	Platform	60+ MHz
Frame Size	8 or 16 bit	1–512 bit	1–4096 bit	4–16 bit	4–16 bit
HW CS Lines	1 (NSS)	4	4 (PCS)	Config	6 (PCS)
TX/RX FIFO	None (1 DR)	64-byte	4 words	SW	4 entries
DMA	Yes	Yes	Yes (EDMA)	Yes	Yes (eDMA)
CRC HW	Yes	No	Yes	No	No
I2S	Yes	Separate	No	No	No
Delay Control	No	Per-trans	CCR regs	Config	CTAR (ns)

7.2 Best Practices

- Verify CPOL/CPHA from the peripheral datasheet — do not assume Mode 0.
- For STM32F4: always read SPI_DR as uint8_t in 8-bit mode to avoid FIFO misalignment.
- For LPSPI: always clear MEN before changing CCR, TCR, or CFGR1 registers.
- For DSPI: always set EOQ=1 on the last PUSHR write to correctly deassert PCS.
- For ESP32: DMA buffers must be in MALLOC_CAP_DMA memory (32-bit aligned, internal SRAM).
- Enable DMA for transfers > 16 bytes to prevent CPU busy-waiting.
- In AUTOSAR: never access Can/Spi driver registers directly — always go through the MCAL API.

7.3 Common Pitfalls

Symptom	Likely Cause	Fix
Data garbled / phase error	Wrong CPOL or CPHA	Scope SCLK/MOSI/MISO, match device datasheet mode
MISO always 0xFF	GPIO not configured as AF	Check MODER, AFR (STM32) or iomux (ESP32/NXP)
MODF on STM32	NSS went low in master	Set SSM+SSI or drive NSS output high
LPSPI no transfer	MEN=0 or TCR not written	Enable module last; write TCR before each frame

DSPI PCS stays asserted	EOQ never set	Set EOQ=1 on final PUSHR write
ESP32 DMA fail	Buffer not DMA-capable	Use heap_caps_malloc(size,MALLOC_CAP_DMA)
AUTOSAR Seq pending forever	ISR not wired to Spi_Isr	Check OS ISR binding in EcucConfig

8. Appendix

8.1 GPIO Quick Reference

Platform	MOSI	MISO	SCLK	CS	Config
STM32F4 SPI1	PA7 (AF5)	PA6 (AF5)	PA5 (AF5)	PA4 GPIO	MODER+AFR[0]
STM32F4 SPI2	PB15 (AF5)	PB14 (AF5)	PB13 (AF5)	PB12 GPIO	MODER+AFR[1]
ESP32 SPI2	GPIO23	GPIO19	GPIO18	GPIO5	spi_bus_config_t
S32K1 LPSPI0	PTB4	PTB5	PTB2	PTB3	PINS_DRV_Init
KV5x SPI0	PTD2 (ALT2)	PTD3 (ALT2)	PTD1 (ALT2)	PTD0 (ALT2)	PORT_PCR mux=2

8.2 References

- STM32F4xx Reference Manual RM0090 — STMicroelectronics
- ESP-IDF Programming Guide, SPI Master/Slave Driver — Espressif Systems
- S32K1xx Reference Manual S32K1XXRM — NXP Semiconductors
- AUTOSAR_SWS_SPIHandlerDriver.pdf — AUTOSAR Consortium
- KV5x Reference Manual KV5XP144M240RM — NXP Semiconductors